Three Address Code into target machine instructions:

Aim:

To write a C program to implement a simple code generator that converts a given Three Address Code into target machine instructions. And to generate appropriate MOV, ADD, SUB, MUL, and DIV instructions.

Algorithm:

1. Start the program.
2. Initialize the register counter `i = 0`.
3. Read a line of Three Address Code (TAC) input.
4. If the input is `"END"`, terminate the program.
5. If the input is an arithmetic expression (`res = op1 oper op2`), generate assembly instructions using register `R[i]` and increment `i`.
6. If the input is a simple assignment (`res = op1`), generate a direct `MOV` instruction.
7. Repeat from step 3 until `"END"` is encountered.
8. Stop the program.

Read input line.

- If input is `"END"`, stop.
- If arithmetic instruction (`res = op1 op op2`), output target code (`MOV`, `ADD/SUB/MUL/DIV`, `MOV`) using register `R[i]` and increment `i`.
- Else if assignment instruction (`res = op1`), output `MOV res, op1`.
- Else, output error message.
- Repeat from Step 1.

Code:

```
#include <stdio.h>
#include <string.h>
```

```
int main()
{
   char result[20], op1[20],
op2[20], oper;
   char tac[100];
   int i = 0;

   printf("Enter Three
Address Code:\n");
   printf("Enter each
statement on a separate
line.\n");
   printf("Enter END to
finish.\n\n");

   printf("Target Code:\n");

   while (1)
   {
      if (fgets(tac, sizeof(tac),
stdin) == NULL)
         break;

      if (strncmp(tac, "END",
3) == 0)
         break;

      /* Arithmetic
instruction (e.g., t1 = a + b)
*/
      if (sscanf(tac, "%19s =
%19s %c %19s", result,
op1, &oper, op2) == 4)
      {
```

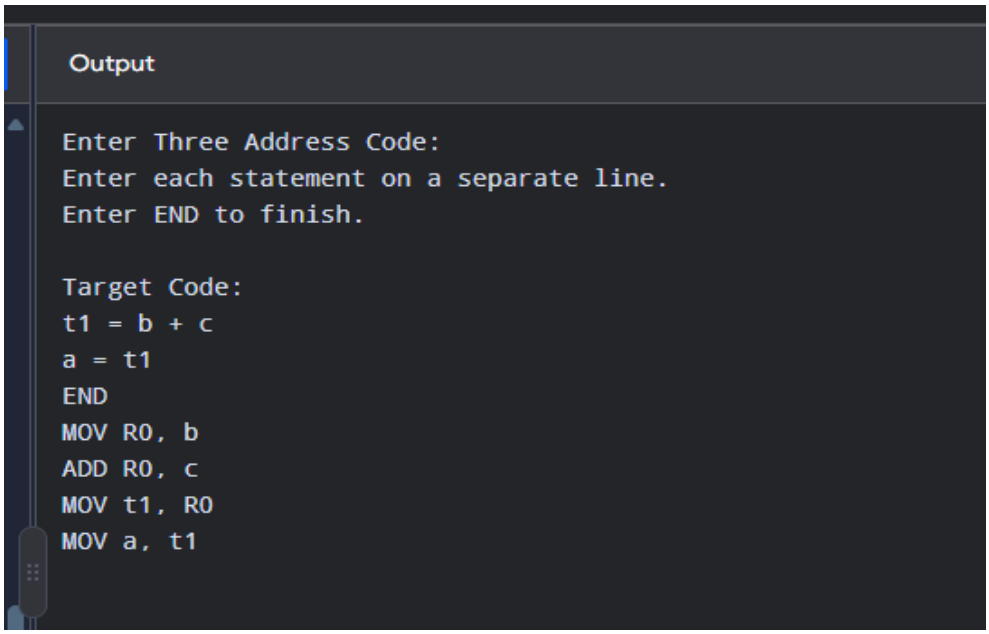
```
        printf("MOV R%d,
%s\n", i, op1);

        switch (oper)
        {
            case '+':
                printf("ADD
R%d, %s\n", i, op2);
                break;
            case '-':
                printf("SUB
R%d, %s\n", i, op2);
                break;
            case '*':
                printf("MUL
R%d, %s\n", i, op2);
                break;
            case '/':
                printf("DIV
R%d, %s\n", i, op2);
                break;
            default:
                printf("; Invalid
operator '%c'\n", oper);
                continue;
        }
        printf("MOV %s,
R%d\n", result, i);
        i++; /* Increment
register index only when a
register is used */
    }
    /* Simple assignment
instruction (e.g., x = t1) */
    else if (sscanf(tac,
```

```
"%19s = %19s", result, op1)
== 2)
    {
        printf("MOV %s,
%s\n", result, op1);
    }
    else
    {
        printf("; Invalid
Three Address Code
format\n");
    }
  }

  return 0;
}
```

OUTPUT:



Result:

Thus, the C program to convert Three Address Code (TAC) into Target Code (Assembly Language) was successfully written, executed, and the output was verified**.**